

BİREYSEL ÜRÜN GELİŞTİRME PROJESİ RAPORU

Dersin Adı: Akademik Yapay Zekaya Giriş

Proje Seçeneği: SEÇENEK 1 - RAG Tabanlı Uzman Bilgi Asistanı

Proje Adı: Akademik Yönetmelik ve Ders Geçme Kriterleri Akıllı Asistanı

Hazırlayan: Elanur CEYLAN – 2544001007

1.PROJE ÖZETİ

Bu projede, öğrencilerin üniversite mevzuatlarına, ders geçme yönetmeliklerine ve yaz okulu kurallarına erişimini kolaylaştırmak amacıyla yerel bir "Doküman Tabanlı Yapay Zekâ Asistanı" (RAG - Retrieval-Augmented Generation) geliştirilmiştir. Geliştirilen sistem, kurumsal verilerin dış kaynaklarla veya üçüncü parti bulut API'leriyle paylaşılmasını önleyecek şekilde tamamen yerel (local) mimaride kurgulanmıştır. Sistem, yüklenen 5 adet kurumsal dokümanı anlamsal parçalara ayırmakta, vektör veritabanına indekslemekte ve LLM (Büyük Dil Modeli) entegrasyonu ile kullanıcının sorularına doğrudan dökümandan beslenen, doğrulanabilir ve kaynak gösteren cevaplar üretmektedir.

2. PROBLEM TANIMI VE GEREKSİNİM ANALİZİ

Mevcut Problem: Üniversite yönetim sistemlerinde yer alan yönetmelikler, onlarca sayfadan oluşan karmaşık hukuki ve akademik metinlerdir. Öğrenciler vize/final oranları, devamsızlık sınırları veya yaz okulu AKTS limitleri gibi kritik bilgilere acil ihtiyaç duyduklarında, bu dökümanlar içerisinde manuel arama yapmakta zorlanmakta ve zaman kaybetmektedir.

Çözüm Yaklaşımı: Bu probleme çözüm olarak, arama motorlarının ötesine geçen, metni anlamsal olarak kavrayan ve öğrenciye doğrudan aradığı spesifik yanıtı sunan bir RAG sistemi tasarlanmıştır.

Hata Yönetimi Gereksinimi (Hallucination Önleme): Standart yapay zekâ modelleri, bilmedikleri konular hakkında uydurma (hallucination) cevaplar üretebilmektedir. Akademik hususlarda uydurma bilgi verilmesi kabul edilemez olduğundan, sistemin dökümanda olmayan sorulara kesinlikle "Bu bilgi kaynak dökümanlarda bulunmamaktadır" yanıtını vermesi temel bir gereksinim olarak belirlenmiştir.

3. KULLANILAN TEKNOLOJİLER VE SEÇİM NEDENLERİ

Python: Projenin ana programlama dili olarak esnekliği ve geniş yapay zekâ kütüphanesi desteği nedeniyle tercih edilmiştir.

Streamlit: Hızlı, kullanıcı dostu ve web tabanlı bir arayüzü minimal kod eforuyla geliştirmek amacıyla seçilmiştir.

LangChain & LangChain-Community: Doküman yükleme, metin parçalama ve vektör arama zincirlerini profesyonel standartlarda yönetmek için kullanılmıştır.

FAISS (Facebook AI Similarity Search): Bellek içi (in-memory) çalışan, son derece hızlı ve yerel mimariler için optimize edilmiş bir vektör veritabanı kütüphanesi olduğu için tercih edilmiştir.

Sentence-Transformers (all-MiniLM-L6-v2): Dokümanları ve kullanıcı sorularını 384 boyutlu anlamsal vektörlere (embedding) dönüştüren, işlemciyi yormayan hafif ve performanslı bir açık kaynaklı modeldir.

HuggingFace Transformers (google/flan-t5-base): Bilgisayarda yerel olarak çalışabilen, ticari lisans kısıtlaması olmayan, metin üretim kapasitesi yüksek (~990 MB boyutunda) bir açık kaynaklı dil modelidir (LLM).

SİSTEM MİMARİSİ VE METODOLOJİ (RAG AKIŞI)

- 1- Veri Toplama (Data Ingestion):** veriler/ klasörü altındaki kaynak metinler (.txt ve .pdf) TextLoader ve PyPDFLoader bileşenleri yardımıyla sisteme dahil edilir. Her dokümanın adı, metadata olarak saklanır.
- 2- Metin Parçalama (Text Splitting):** Metinler anlamsal bütünlüğünü kaybetmemesi adına RecursiveCharacterTextSplitter ile CHUNK_SIZE = 1000 karakter uzunluğunda ve CHUNK_OVERLAP = 150 karakter örtüşme payı ile bölünür.
- 3- Vektörleştirme ve İndeksleme:** Ayrılan metin parçaları all-MiniLM-L6-v2 modeli ile matematiksel vektörlere dönüştürülerek FAISS veritabanına kaydedilir.
- 4- Anlamsal Arama (Semantic Search):** Kullanıcı bir soru sorduğunda, soru vektörleştirilir ve FAISS üzerinde similarity_search_with_relevance_scores fonksiyonu kullanılarak en yakın TOP_K = 4 metin parçası çağrılır.
- 5- Eşik Değer Filtrelemesi (Hata Yönetimi):** Çekilen parçaların benzerlik skoru BENZERLIK_ESIGI = 0.45 değerinin altındaysa, arama otomatik olarak kesilir ve uydurma cevap üretimi engellenir.
- 6- Cevap Üretimi (Generation):** Seçilen doküman parçaları ve kullanıcı sorusu, özel bir prompt yapısı ile flan-t5-base modeline beslenir ve nihai yanıt üretilerek arayüzde alt tarafta kaynağıyla birlikte listelenir.

5. TEST SENARYOLARI VE DOĞRULAMA (EVALUATION)

Geliştirilen sistem üzerinde iki temel senaryo simüle edilerek sistemin doğruluğu test edilmiştir:

Senaryo A (Başarılı Bilgi Getirimi):

Soru: "Yaz okulunda en fazla kaç ders seçebilirim?"

Sistem Davranışı: FAISS veritabanı yaz_okulu_yonetmeligi.txt dokümanını başarıyla tespit etmiş, LLM modeli metinden beslenerek "En fazla 3 ders veya 20 AKTS seçilebilir" yanıtını üretmiştir. Arayüzde kaynak olarak ilgili doküman adı başarıyla gösterilmiştir.

Senaryo B (Hata Yönetimi ve Hallucination Engelleme):

Soru: "Kampüsteki yemekhane saat kaçta açılıyor?"

Sistem Davranışı: Yüklenen 5 dokümanda yemekhane saatine dair bir veri bulunmadığından, benzerlik skoru 0.45 sınırının altında kalmıştır. Sistem LLM modeline gitmeden doğrudan

filtreye takılmış ve ekrana güvenli mesaj olan "Bu bilgi yüklenen kaynak dokümanlarda bulunmamaktadır." uyarısını basmıştır.

6. SİSTEMİN GÜÇLÜ VE SINIRLI YÖNLERİ

Güçlü Yönler:

- 1- %100 Veri Güvenliği: İnternet veya dış kaynaklı bir API (OpenAI, Anthropic vb.) kullanılmadığı için kurumsal dökümanlar tamamen yerel bilgisayar ortamında kalır.
- 2- Sıfır Maliyet: Kullanılan tüm kütüphaneler ve yapay zekâ modelleri tamamen açık kaynaklı ve ücretsizdir.
- 3- Güvenli Hata Yönetimi: Geliştirilen benzerlik eşiği mekanizması sayesinde sistemin uydurma bilgi üretmesi kesin olarak engellenmiştir.

Sınırlı Yönler / Gelecek Çalışmalar:

- 1- Yerel donanım bağımlılığı (Yüksek CPU/RAM ihtiyacı). Model ilk çalışırken yerel kütüphane bağımlılıkları nedeniyle donanım kaynaklarını yoğun tüketmektedir. Gelecek çalışmalarda sistem Dockerize edilerek bulut ortamlarında (Örn: AWS, Google Cloud) mikroservis olarak ölçeklenecektir.

7. PROJE YÖNETİMİ VE SÜRÜM BİLGİLERİ

GitHub Issues İle Süreç Yönetimi: Proje geliştirme aşamaları (Gereksinim analizi, model seçimi, arayüz tasarımı, testler) GitHub Issues mimarisi üzerinde tasklara bölünmüş, süreç takibi resmi süreç yönetim metodolojilerine uygun olarak tamamlanmış ve kapatılmıştır.

Semantik Sürümleme (Semantic Versioning): Projenin tüm testleri tamamlanmış, kararlı ve sunuma hazır ilk versiyonu GitHub deponun "Releases" bölümünden resmi standartlara uygun olarak **v1.0.0** etiketiyle (tag) dökümanite edilerek yayımlanmıştır.